

LAPORAN PRATIUM STRUKTUR DATA

**IMPLEMENTASI STRUKTUR POHON BINER (BINARY TREE) DAN
TRAVERSAL GRAF (BFS/DFS) DALAM JAVA DENGAN VISUALISASI PETA
OUT OF THE BOX**

DI SUSUN OLEH :

ABDUL KARIM ALGAZALI

NIM 2511532029

DOSEN PENGAMPU : Dr.WAHYUDI, S.T, M.T

ASISTEN LABORATORIUM : Zahran Azaria Anvaya



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

PADANG

2026

KATA PENGANTAR

Puji syukur kehadiran Tuhan Yang Maha Esa, karena atas rahmat dan izin-Nya laporan praktikum "Implementasi Struktur Pohon Biner (Binary Tree) dan Traversal Graf (BFS/DFS) dalam Java dengan Visualisasi Peta Out of The Box" ini dapat diselesaikan dengan baik. Saya ucapkan terima kasih sebesar-besarnya kepada Dr. Wahyudi, S.T, M.T selaku dosen pengampu mata kuliah Struktur Data, serta kepada asisten laboratorium yang telah membimbing pelaksanaan praktikum ini.

Laporan ini disusun sebagai bentuk dokumentasi kegiatan praktikum Pekan 9 yang bertujuan untuk memahami dan mengimplementasikan dua topik utama: struktur data pohon biner (*binary tree*) beserta metode traversal (preorder, inorder, postorder) serta algoritma pencarian pada graf (DFS dan BFS). Berbeda dengan praktikum pekan 8 yang fokus pada algoritma sorting lanjutan, praktikum kali ini menekankan pada representasi data hirarkis (pohon) dan hubungan antar simpul (graf). Selain implementasi command line, praktikum ini juga menuntut pembuatan program visual interaktif untuk BFS dan DFS dengan tema unik "Labirin Bawah Laut & Stasiun Luar Angkasa" menggunakan Java Swing.

Saya menyadari bahwa penulisan laporan praktikum ini masih jauh dari kata sempurna. Namun, dengan segala kemampuan dan pengetahuan yang dimiliki, saya telah berupaya agar laporan ini dapat selesai dengan baik. Oleh karenanya, saya dengan rendah hati menerima saran, masukan, dan kritikan guna penyempurnaan laporan ini.

Padang, Mei 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI.....	ii
BAB I PENDAHULUAN.....	3
1.1 Latar Belakang.....	3
1.2 Tujuan	3
1.3 Manfaat	4
BAB II PEMBAHASAN.....	5
2.1 Konsep Algoritma Sorting.....	5
2.2 Langkah pengerjaan.....	6
2.3 Latihan	21
BAB III KESIMPULAN.....	24
3.1 Kesimpulan	24
3.2 Saran	24
DAFTAR PUSTAKA.....	25

BAB I

PENDAHULUAN

1.1 Latar Belakang

Pada praktikum minggu ke-8, kita telah mempelajari algoritma sorting lanjutan (Merge Sort, Quick Sort, Shell Sort) yang berfokus pada pengurutan data linear. Memasuki minggu ke-9, topik yang dipelajari beralih ke struktur data non-linear yaitu pohon biner (binary tree) dan graf (graph). Kedua struktur data ini sangat penting karena mampu merepresentasikan hubungan hierarkis (pohon) dan hubungan kompleks antar entitas (graf) yang banyak ditemui dalam kehidupan nyata, seperti sistem file, jaringan sosial, peta navigasi, dan lain-lain.

Praktikum minggu ke-9 terdiri dari beberapa tahap:

1. Implementasi Binary Tree – Membangun kelas Node dan Binary Tree, kemudian melakukan traversal Preorder, Inorder, Postorder secara rekursif, serta menghitung jumlah simpul.
2. Traversal Graf (DFS dan BFS) – Membangun graf tak berarah dengan adjacency list, lalu mengimplementasikan pencarian Depth First Search (DFS) rekursif dan Breadth First Search (BFS) iteratif.
3. Tugas Besar: Visualisasi Peta "Out of The Box" – Membuat aplikasi GUI interaktif dengan Java Swing yang menggabungkan konsep BFS dan DFS pada graf yang bertema gabungan stasiun luar angkasa dan kota bawah laut. Program ini menampilkan visualisasi node, edge, warna node berdasarkan status kunjungan, serta jalur yang ditemukan dari start ke goal.

Seluruh program dalam praktikum ini menggunakan penamaan variabel yang diakhiri dengan NIM (2029) sebagai identitas penulis. Tugas besar juga mensyaratkan minimal 10 vertex dan 15 edge, serta background putih untuk kenyamanan visual.

1.2 Tujuan

1. Memahami konsep dan mekanisme tiga algoritma sorting lanjutan: Merge Sort, Memahami konsep struktur data pohon biner (binary tree), termasuk simpul (node), akar (root), anak kiri dan kanan (left/right child).
2. Mampu mengimplementasikan operasi dasar pada binary tree: penambahan simpul, pencarian, perhitungan jumlah simpul, dan tiga metode traversal (preorder, inorder, postorder).
3. Memahami perbedaan antara struktur pohon dan graf, khususnya pada graf yang memungkinkan adanya siklus dan multiple path.
4. Mampu mengimplementasikan algoritma penelusuran graf: Depth First Search (DFS) secara rekursif dan Breadth First Search (BFS) menggunakan queue.
5. Mampu membangun program GUI interaktif dengan Java Swing untuk memvisualisasikan graf dan proses pencarian jalur menggunakan BFS/DFS, termasuk menampilkan node yang dikunjungi, jalur terpendek (BFS) atau jalur pertama (DFS), serta warna yang berbeda untuk start, goal, node tervisit, dan node jalur.
6. Menerapkan tema yang kreatif dan "out of the box" (luar angkasa dan bawah laut) dengan minimal 10 vertex dan 15 edge.

1.3 Manfaat

1. Mahasiswa mampu membedakan implementasi struktur data linear dan non-linear, serta memilih struktur yang tepat untuk permasalahan tertentu.
2. Mahasiswa terampil mengimplementasikan operasi rekursif pada pohon biner (traversal, penghitungan) yang menjadi fondasi untuk struktur data lanjutan seperti binary search tree (BST), heap, dan AVL tree.
3. Mahasiswa memahami mekanisme BFS (pencarian level-by-level) dan DFS (pencarian mendalam terlebih dahulu) serta kelebihan dan kekurangan masing-masing.
4. Mahasiswa mampu mengintegrasikan algoritma graf dengan antarmuka pengguna grafis, termasuk visualisasi langsung dari state algoritma (node dikunjungi, jalur ditemukan).
5. Mahasiswa memperoleh pengalaman dalam pengembangan aplikasi berbasis event-driven dengan tema yang menarik, yang dapat dikembangkan lebih lanjut untuk aplikasi navigasi atau game puzzle.

BAB II

PEMBAHASAN

2.1 Konsep Algoritma Sorting

a. Pohon Biner (Binary Tree)

Pohon biner adalah struktur data hierarkis di mana setiap simpul (node) paling banyak memiliki dua anak, yaitu anak kiri (left child) dan anak kanan (right child). Simpul paling atas disebut akar (root). Pohon biner memiliki beberapa metode traversal:

- Preorder: Kunjungi akar, lalu subtree kiri secara rekursif, lalu subtree kanan (Root → Left → Right).
- Inorder: Kunjungi subtree kiri, lalu akar, lalu subtree kanan (Left → Root → Right). Pada Binary Search Tree (BST), inorder menghasilkan urutan terurut.
- Postorder: Kunjungi subtree kiri, lalu subtree kanan, lalu akar (Left → Right → Root).

Operasi lain yang umum: pencarian nilai, penghitungan jumlah simpul, pengecekan apakah pohon kosong.

b. Graf (Graph)

Graf adalah struktur data yang terdiri dari himpunan simpul (vertex) dan sisi (edge) yang menghubungkan pasangan simpul. Graf dapat berarah (directed) atau tak berarah (undirected). Dua algoritma penelusuran graf yang paling dasar:

- BFS (Breadth First Search) : Menggunakan queue, menjelajahi semua simpul pada level yang sama sebelum berpindah ke level berikutnya. BFS dapat menemukan jalur terpendek (dalam jumlah sisi) pada graf tak berbobot.
- DFS (Depth First Search) : Menggunakan stack (atau rekursi), menjelajahi sedalam mungkin pada satu cabang sebelum mundur (backtrack). DFS menggunakan memori lebih kecil tetapi tidak menjamin jalur terpendek.

Dalam praktikum ini, graf direpresentasikan dengan adjacency list (Map<String, List<String>>). Untuk visualisasi, digunakan koordinat tetap agar tampilan konsisten.

2.2 Langkah pengerjaan

1. Program Node_2511532029

Program:

```
1 package pekang_2511532029;
2
3 public class Node_2511532029 {
4     int data_2029;
5     Node_2511532029 left_2029;
6     Node_2511532029 right_2029;
7
8     public Node_2511532029 (int data_2029) {
9         this.data_2029 = data_2029;
10        left_2029 = null;
11        right_2029 = null;
12    }
13
14    public void setLeft_2029 (Node_2511532029 node) {
15        if (left_2029 == null) {
16            left_2029 = node;
17        }
18    }
19
20    public void setRight_2029 (Node_2511532029 node) {
21        if (right_2029 == null) {
22            right_2029 = node;
23        }
24    }
25
26    public Node_2511532029 getLeft_2029 () {
27        return this.left_2029;
28    }
29
30    public Node_2511532029 getRight_2029 () {
31        return this.right_2029;
32    }
33
34    public int getData_2029 () {
35        return data_2029;
36    }
37
38    public void setData_2029 (int data_2029) {
39        this.data_2029 = data_2029;
40    }
41
42    void printPreorder_2029 (Node_2511532029 node) {
43        if (node == null) {
44            return;
45        }
46        System.out.print(node.data_2029 + " ");
47        printPreorder_2029 (node.left_2029);
48        printPreorder_2029 (node.right_2029);
49    }
50
51    void printPostorder_2029 (Node_2511532029 node) {
52        if (node == null) {
53            return;
54        }
55        printPostorder_2029 (node.left_2029);
56        printPostorder_2029 (node.right_2029);
57        System.out.print(node.data_2029 + " ");
58    }
59
60    void printInorder_2029 (Node_2511532029 node) {
61        if (node == null) {
62            return;
63        }
64        printInorder_2029 (node.left_2029);
65        System.out.print(node.data_2029 + " ");
66        printInorder_2029 (node.right_2029);
67    }
68
69    public String print () {
70        return this.print (prefix: "", isTail: true, sb: "");
71    }
72
73    public String print (String prefix, boolean isTail, String sb) {
74        if (right_2029 != null) {
75            right_2029.print (prefix + (isTail ? " " : " "), isTail: false, sb);
76        }
77        System.out.print (prefix + (isTail ? " | " : " / ") + data_2029);
78        if (left_2029 != null) {
79            left_2029.print (prefix + (isTail ? " " : " "), isTail: true, sb);
80        }
81        return sb;
82    }
83 }
```

Gambar 2.1

Penjelasan program:

Kelas Node_2511532029 merepresentasikan satu simpul dalam pohon biner. Setiap simpul menyimpan data integer, serta referensi ke anak kiri dan kanan. Konstruktor menginisialisasi data dan kedua anak dengan null. Metode setLeft_2029 dan setRight_2029 hanya mengisi jika anak masih kosong (menghindari penimpaan). Metode traversal (printPreorder_2029, printInorder_2029, printPostorder_2029) diimplementasikan secara rekursif. Metode print() dengan parameter prefix digunakan untuk mencetak struktur pohon secara visual (mirip dengan perintah tree di Linux). Kelas ini menjadi blok bangunan dasar untuk BTree_2511532029.

2. BTree_2511532029 Program:

```

1 //Kelas BTree_2511532029
2
3 public class BTree_2511532029 {
4     private Node_2511532029 root;
5     private Node_2511532029 currentNode;
6     public BTree_2511532029() {
7         root = null;
8     }
9     public boolean search_2029(int data_2029) {
10         return search_2029(root, data_2029);
11     }
12     private boolean search_2029(Node_2511532029 node, int data_2029) {
13         if (node.getData_2029() == data_2029) {
14             return true;
15         }
16         if (node.getLeft_2029() != null) {
17             if (search_2029(node.getLeft_2029(), data_2029)) {
18                 return true;
19             }
20         }
21         if (node.getRight_2029() != null) {
22             if (search_2029(node.getRight_2029(), data_2029)) {
23                 return true;
24             }
25         }
26         return false;
27     }
28     public void printInorder_2029() {
29         root.printInorder_2029();
30     }
31     public void printPreorder_2029() {
32         root.printPreorder_2029();
33     }
34     public void printPostorder_2029() {
35         root.printPostorder_2029();
36     }
37     public Node_2511532029 getRoot() {
38         return root;
39     }
40     public boolean isEmpty() {
41         return root == null;
42     }
43     public int countNodes() {
44         return countNodes(root);
45     }
46     private int countNodes(Node_2511532029 node) {
47         int count = 1;
48         if (node == null) {
49             return 0;
50         }
51         count += countNodes(node.getLeft_2029());
52         count += countNodes(node.getRight_2029());
53         return count;
54     }
55     public void print() {
56         root.print();
57     }
58     public Node_2511532029 getCurrent() {
59         return currentNode;
60     }
61     public void setCurrent(Node_2511532029 node) {
62         this.currentNode = node;
63     }
64     public void setRoot(Node_2511532029 root) {
65         this.root = root;
66     }
67 }

```

Gambar 2.2

Penjelasan program:

Kelas BTree_2511532029 berfungsi sebagai wrapper untuk mengelola pohon biner. Atribut root menyimpan akar pohon, dan currentNode menyimpan simpul aktif (digunakan di driver). Metode search_2029 melakukan pencarian nilai secara rekursif dengan mengecek node saat ini, lalu rekursif ke kiri dan kanan. Metode countNodes juga rekursif dengan basis jika node null maka 0, jika tidak maka 1 + jumlah node kiri + jumlah node kanan. Metode print memanggil metode visual dari node root. Kelas ini menyediakan antarmuka yang lebih rapi daripada langsung menggunakan Node.

3. Program BTreeDriver_2511532029

Program:

```
1 package pekam_2511532029;
2
3 public class BTreeDriver_2511532029 {
4     public static void main(String[] args) {
5         // Membuat pohon
6         BTree_2511532029 tree_2029 = new BTree_2511532029();
7         System.out.println("Jumlah simpul awal pohon: ");
8         System.out.println(tree_2029.countNodes());
9
10        // Menambahkan simpul data 1
11        Node_2511532029 root_2029 = new Node_2511532029(data_2029: 1);
12
13        // Menjadikan simpul 1 sebagai root
14        tree_2029.setRoot(root_2029);
15        System.out.println("Jumlah simpul jika hanya ada root: ");
16        System.out.println(tree_2029.countNodes());
17
18        Node_2511532029 node2 = new Node_2511532029(data_2029: 2);
19        Node_2511532029 node3 = new Node_2511532029(data_2029: 3);
20        Node_2511532029 node4 = new Node_2511532029(data_2029: 4);
21        Node_2511532029 node5 = new Node_2511532029(data_2029: 5);
22        Node_2511532029 node6 = new Node_2511532029(data_2029: 6);
23        Node_2511532029 node7 = new Node_2511532029(data_2029: 7);
24        Node_2511532029 node8 = new Node_2511532029(data_2029: 8);
25        Node_2511532029 node9 = new Node_2511532029(data_2029: 9);
26
27        root_2029.setLeft_2029(node2);
28        node2.setLeft_2029(node4);
29        node2.setRight_2029(node5);
30        node4.setRight_2029(node6);
31        root_2029.setRight_2029(node3);
32        node3.setLeft_2029(node6);
33        node3.setRight_2029(node7);
34        node6.setLeft_2029(node9);
35
36        // Set root ke currentNode
37        tree_2029.setCurrentNode(tree_2029.getRoot());
38        System.out.println("Menampilkan simpul terakhir: ");
39        System.out.println(tree_2029.getCurrent().getData_2029());
40
41        System.out.println("Jumlah simpul: setelah simpul 7 ditambahkan");
42        System.out.println(tree_2029.countNodes());
43
44        System.out.println("InOrder: ");
45        tree_2029.printInorder_2029();
46
47        System.out.println("Preorder: ");
48        tree_2029.printPreorder_2029();
49
50        System.out.println("Postorder: ");
51        tree_2029.printPostorder_2029();
52
53        System.out.println("Menampilkan simpul dalam bentuk pohon");
54        tree_2029.print();
55    }
56 }
```

Gambar 2.3

Penjelasan program:

Driver ini membangun pohon biner dengan struktur sebagai berikut (angka menunjukkan data):

text

```
1
/\
2 3
/\ /\
4 5 6 7
\ /
8 9
```

Proses:

- Awalnya pohon kosong, countNodes() mengembalikan 0.
- Setelah root (1) ditambahkan, jumlah simpul = 1.
- Selanjutnya menambahkan node 2,3,4,5,6,7,8,9 sesuai dengan relasi parent-child.
- Ditampilkan simpul current (root) yaitu 1.
- countNodes() setelah semua node ditambahkan mengembalikan 9.
- Dicetak hasil traversal Inorder, Preorder, Postorder.
- Terakhir, metode print() menampilkan representasi grafis pohon.

Output:

```
PS D:\actual_eclipse> & "C:\Program Files\Java\
Jumlah simpul awal pohon: 0
Jumlah simpul jika hanya ada root: 1
menampilkan simpul terakhir:
1
Jumlah simpul; setelah simpul 7 ditambahkan
9
InOrder:
482519637
Preorder:
124853697
Postorder :
845296731
Menampilkan simpul dalam bentuk pohon
| /--7
| /--3
| | \--6
| | \--9
|--1
| /--5
|--2
| /--8
|--4
PS D:\actual_eclipse>
```

Gambar 2.4

4. Program GraphTraversal_2511532029

Program :

```
1 package pekam_2511532029;
2
3 import java.util.*;
4
5 public class GraphTraversal_2511532029 {
6     private Map<String, List<String>> graph_2029 = new HashMap<>();
7
8     // Menambahkan edge (graf tak berarah)
9     public void addEdge_2029(String node1_2029, String node2_2029) {
10         graph_2029.putIfAbsent(node1_2029, new ArrayList<>());
11         graph_2029.putIfAbsent(node2_2029, new ArrayList<>());
12         graph_2029.get(node1_2029).add(node2_2029);
13         graph_2029.get(node2_2029).add(node1_2029);
14     }
15
16     // Menampilkan graf awal
17     public void printGraph_2029() {
18         System.out.println("Graf Awal (Adjacency List):");
19         for (String node_2029 : graph_2029.keySet()) {
20             System.out.print(node_2029 + " : ");
21             List<String> neighbors_2029 = graph_2029.get(node_2029);
22             System.out.println(String.join(", ", neighbors_2029));
23         }
24         System.out.println();
25     }
26
27     // DFS rekursif
28     public void dfs_2029(String start_2029) {
29         Set<String> visited_2029 = new HashSet<>();
30         System.out.println("Pemelusuran DFS:");
31         dfsHelper_2029(start_2029, visited_2029);
32         System.out.println();
33     }
34
35     private void dfsHelper_2029(String current_2029, Set<String> visited_2029) {
36         if (visited_2029.contains(current_2029)) return;
37
38         visited_2029.add(current_2029);
39         System.out.print(current_2029 + " ");
40
41         for (String neighbor_2029 : graph_2029.getOrDefault(current_2029, new ArrayList<>())) {
42             dfsHelper_2029(neighbor_2029, visited_2029);
43         }
44     }
45
46     // BFS iteratif
47     public void bfs_2029(String start_2029) {
48         Set<String> visited_2029 = new HashSet<>();
49         Queue<String> queue_2029 = new LinkedList<>();
50
51         queue_2029.add(start_2029);
52         visited_2029.add(start_2029);
53
54         System.out.println("Pemelusuran BFS:");
55         while (!queue_2029.isEmpty()) {
56             String current_2029 = queue_2029.poll();
57             System.out.print(current_2029 + " ");
58
59             for (String neighbor_2029 : graph_2029.getOrDefault(current_2029, new ArrayList<>())) {
60                 if (!visited_2029.contains(neighbor_2029)) {
61                     queue_2029.add(neighbor_2029);
62                     visited_2029.add(neighbor_2029);
63                 }
64             }
65         }
66         System.out.println();
67     }
68
69     // Main
70     @SuppressWarnings("Run")
71     public static void main(String[] args_2029) {
72         GraphTraversal_2511532029 graph_2029 = new GraphTraversal_2511532029();
73
74         // Contoh graf: A-B, A-C, B-D, B-E
75         graph_2029.addEdge_2029("A", "B", "A", "B");
76         graph_2029.addEdge_2029("A", "C", "A", "C");
77         graph_2029.addEdge_2029("B", "D", "B", "D");
78         graph_2029.addEdge_2029("B", "E", "B", "E");
79
80         // Cetak graf awal
81         System.out.println("Graf Awal adalah:");
82         graph_2029.printGraph_2029();
83
84         // Lakukan penelusuran
85         graph_2029.dfs_2029("A");
86         graph_2029.bfs_2029("A");
87     }
88 }
```

Gambar 2.5

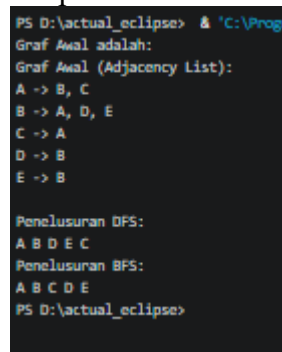
Penjelasan program:

Program ini membuat graf tak berarah dengan 5 simpul (A, B, C, D, E). Struktur graf:

- A terhubung ke B dan C.
- B terhubung ke A, D, E.
- C terhubung ke A.
- D terhubung ke B.
- E terhubung ke B.

Metode `addEdge_2029` menambahkan sisi dua arah. `printGraph_2029` menampilkan adjacency list. DFS diimplementasikan dengan rekursi: fungsi `dfsHelper` menandai node sebagai visited, mencetak, lalu memanggil dirinya untuk setiap tetangga yang belum dikunjungi. BFS menggunakan queue: node awal dimasukkan ke queue, lalu selama queue tidak kosong, ambil node, cetak, dan masukkan semua tetangga yang belum dikunjungi.

Output:



```
PS D:\actual_eclipse> & "C:\Progr
Graf Awal adalah:
Graf Awal (Adjacency List):
A -> B, C
B -> A, D, E
C -> A
D -> B
E -> B

Penelusuran DFS:
A B D E C
Penelusuran BFS:
A B C D E
PS D:\actual_eclipse>
```

Gambar 2.6

2.3 Latihan

Deskripsi Program:

Buatlah program GUI dengan Java Swing yang memvisualisasikan graf dengan tema "Labirin Bawah Laut & Stasiun Luar Angkasa". Graf harus memiliki minimal 10 vertex dan 15 edge. Program harus menyediakan pilihan algoritma BFS dan DFS untuk mencari jalur dari simpul start ke simpul goal, serta menampilkan:

- Node yang dikunjungi (warna biru muda)
- Node yang merupakan bagian dari jalur hasil (warna hijau terang)
- Node start (warna merah) dan node goal (warna oranye)
- Urutan kunjungan dan jalur yang ditemukan pada area teks
- Latar belakang putih agar nyaman dilihat

1. Program Lagu_2511532029

Program\Syntax:

```
package pekan9_2511532029;
```

```
import java.awt.*;
import java.util.*;
```

```

import java.util.List;
import javax.swing.*.*;

/**
 *
 *
 *
 *
 */
public class BFSDFSMap_2511532029 extends JFrame {
    // Data graph
    private List<String> vertices_2029;
    private Map<String, List<String>> adjList_2029;
    private Map<String, Point> nodePositions_2029;

    // GUI components
    private GraphPanel_2029 graphPanel_2029;
    private JComboBox<String> startCombo_2029;
    private JComboBox<String> goalCombo_2029;
    private JTextArea resultArea_2029;

    public BFSDFSMap_2511532029() {
        initGraph_2029();
        initGUI_2029();
        setTitle("Pencarian Jalur - BFS & DFS (Labirin Bawah Laut + Stasiun Luar
Angkasa)");
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        pack();
        setLocationRelativeTo(null);
        setVisible(true);
    }

    /**
     * Graph unik & out of the box:
     * Menggabungkan stasiun luar angkasa dengan kota bawah laut.
     * Minimal 10 vertex, 15 edge.
     */
    private void initGraph_2029() {
        vertices_2029 = Arrays.asList(
            "Neptunus Gateway", // stasiun luar angkasa
            "Mars Colony", // koloni Mars
            "Europa Drill Station", // stasiun pengeboran bulan Europa
            "Titan Floating City", // kota di Titan
            "Atlantis Deep Sea", // kota bawah laut
            "Leviathan Trench", // palung laut dalam
            "Coral Highway", // jalan bawah laut
            "Abyss Lab", // laboratorium kegelapan laut
            "Space Elevator", // penghubung darat-luar angkasa
            "Quantum Portal" // portal fiksi antar-dimensi
        );
    }
}

```

```

);

adjList_2029 = new HashMap<>();
for (String v : vertices_2029) {
    adjList_2029.put(v, new ArrayList<>());
}

// 15 edge (undirected) yang menghubungkan dunia luar angkasa & bawah laut
addEdge_2029("Neptunus Gateway", "Mars Colony");
addEdge_2029("Neptunus Gateway", "Space Elevator");
addEdge_2029("Neptunus Gateway", "Quantum Portal");
addEdge_2029("Mars Colony", "Europa Drill Station");
addEdge_2029("Mars Colony", "Titan Floating City");
addEdge_2029("Europa Drill Station", "Titan Floating City");
addEdge_2029("Europa Drill Station", "Space Elevator");
addEdge_2029("Titan Floating City", "Atlantis Deep Sea");
addEdge_2029("Space Elevator", "Coral Highway");
addEdge_2029("Atlantis Deep Sea", "Leviathan Trench");
addEdge_2029("Atlantis Deep Sea", "Coral Highway");
addEdge_2029("Leviathan Trench", "Abyss Lab");
addEdge_2029("Coral Highway", "Abyss Lab");
addEdge_2029("Coral Highway", "Quantum Portal");
addEdge_2029("Abyss Lab", "Quantum Portal");

// Posisi untuk visualisasi di panel 900x600
nodePositions_2029 = new HashMap<>();
nodePositions_2029.put("Neptunus Gateway", new Point(700, 80));
nodePositions_2029.put("Mars Colony", new Point(550, 150));
nodePositions_2029.put("Europa Drill Station", new Point(400, 220));
nodePositions_2029.put("Titan Floating City", new Point(250, 200));
nodePositions_2029.put("Atlantis Deep Sea", new Point(150, 350));
nodePositions_2029.put("Leviathan Trench", new Point(80, 480));
nodePositions_2029.put("Coral Highway", new Point(300, 450));
nodePositions_2029.put("Abyss Lab", new Point(200, 550));
nodePositions_2029.put("Space Elevator", new Point(500, 400));
nodePositions_2029.put("Quantum Portal", new Point(650, 500));
}

private void addEdge_2029(String u, String v) {
    adjList_2029.get(u).add(v);
    adjList_2029.get(v).add(u);
}

private void initGUI_2029() {
    setLayout(new BorderLayout());

    // Panel kontrol atas
    JPanel controlPanel_2029 = new JPanel(new FlowLayout());
    controlPanel_2029.add(new JLabel("Start:"));

```

```

startCombo_2029 = new JComboBox<>(vertices_2029.toArray(new String[0]));
controlPanel_2029.add(startCombo_2029);
controlPanel_2029.add(new JLabel("Goal:"));
goalCombo_2029 = new JComboBox<>(vertices_2029.toArray(new String[0]));
controlPanel_2029.add(goalCombo_2029);

JButton bfsBtn_2029 = new JButton("BFS");
JButton dfsBtn_2029 = new JButton("DFS");
JButton resetBtn_2029 = new JButton("Reset");
controlPanel_2029.add(bfsBtn_2029);
controlPanel_2029.add(dfsBtn_2029);
controlPanel_2029.add(resetBtn_2029);
add(controlPanel_2029, BorderLayout.NORTH);

// Area visualisasi graph dengan background putih
graphPanel_2029 = new GraphPanel_2029();
graphPanel_2029.setPreferredSize(new Dimension(900, 650));
graphPanel_2029.setBackground(Color.WHITE);
add(graphPanel_2029, BorderLayout.CENTER);

// Area hasil pencarian dengan background putih
resultArea_2029 = new JTextArea(10, 45);
resultArea_2029.setEditable(false);
resultArea_2029.setBackground(Color.WHITE);
resultArea_2029.setForeground(Color.BLACK);
resultArea_2029.setFont(new Font("Monospaced", Font.PLAIN, 12));
JScrollPane scrollPane_2029 = new JScrollPane(resultArea_2029);
add(scrollPane_2029, BorderLayout.SOUTH);

// Event handlers
bfsBtn_2029.addActionListener(e -> runBFS_2029());
dfsBtn_2029.addActionListener(e -> runDFS_2029());
resetBtn_2029.addActionListener(e -> resetGraph_2029());
}

private void runBFS_2029() {
    String start_2029 = (String) startCombo_2029.getSelectedItem();
    String goal_2029 = (String) goalCombo_2029.getSelectedItem();
    if (start_2029 == null || goal_2029 == null) return;

    Queue<String> queue_2029 = new LinkedList<>();
    Set<String> visited_2029 = new HashSet<>();
    Map<String, String> parent_2029 = new HashMap<>();
    List<String> order_2029 = new ArrayList<>();

    queue_2029.add(start_2029);
    visited_2029.add(start_2029);
    parent_2029.put(start_2029, null);

```

```

        boolean found_2029 = false;
        while (!queue_2029.isEmpty()) {
            String cur = queue_2029.poll();
            order_2029.add(cur);
            if (cur.equals(goal_2029)) {
                found_2029 = true;
                break;
            }
            for (String neighbor : adjList_2029.get(cur)) {
                if (!visited_2029.contains(neighbor)) {
                    visited_2029.add(neighbor);
                    parent_2029.put(neighbor, cur);
                    queue_2029.add(neighbor);
                }
            }
        }

        List<String> path_2029 = buildPath_2029(parent_2029, start_2029, goal_2029,
        found_2029);
        displayResult_2029("BFS", order_2029, path_2029, visited_2029.size());
        graphPanel_2029.setVisitedAndPath_2029(visited_2029, path_2029, start_2029,
        goal_2029);
    }

    // Class pembantu untuk DFS agar parent tersimpan akurat
    class NodeInfo_2029 {
        String nodeId;
        String parentId;
        NodeInfo_2029(String nodeId, String parentId) {
            this.nodeId = nodeId;
            this.parentId = parentId;
        }
    }

    private void runDFS_2029() {
        String start_2029 = (String) startCombo_2029.getSelectedItem();
        String goal_2029 = (String) goalCombo_2029.getSelectedItem();
        if (start_2029 == null || goal_2029 == null) return;

        Stack<NodeInfo_2029> stack_2029 = new Stack<>();
        Set<String> visited_2029 = new HashSet<>();
        Map<String, String> parent_2029 = new HashMap<>();
        List<String> order_2029 = new ArrayList<>();

        // Push node pertama dengan parent null
        stack_2029.push(new NodeInfo_2029(start_2029, null));

        boolean found_2029 = false;
        while (!stack_2029.isEmpty()) {

```

```

NodeInfo_2029 curInfo = stack_2029.pop();
String cur = curInfo.nodeId;

if (visited_2029.contains(cur)) continue;

// Catat visited, urutan, dan parent yang benar saat di-pop
visited_2029.add(cur);
order_2029.add(cur);
parent_2029.put(cur, curInfo.parentId);

if (cur.equals(goal_2029)) {
    found_2029 = true;
    break;
}

// Masukkan tetangga ke dalam stack
for (String neighbor : adjList_2029.get(cur)) {
    if (!visited_2029.contains(neighbor)) {
        stack_2029.push(new NodeInfo_2029(neighbor, cur));
    }
}
}

List<String> path_2029 = buildPath_2029(parent_2029, start_2029, goal_2029,
found_2029);
displayResult_2029("DFS", order_2029, path_2029, visited_2029.size());
graphPanel_2029.setVisitedAndPath_2029(visited_2029, path_2029, start_2029,
goal_2029);
}

private List<String> buildPath_2029(Map<String, String> parent, String start, String
goal, boolean found) {
    List<String> path = new ArrayList<>();
    if (!found) return path;
    String node = goal;
    while (node != null) {
        path.add(0, node);
        node = parent.get(node);
    }
    return path;
}

private void displayResult_2029(String algo, List<String> order, List<String> path, int
explored) {
    StringBuilder sb = new StringBuilder();
    sb.append("=== ").append(algo).append(" pada Peta Luar Angkasa & Bawah Laut
===\n");
    sb.append("Urutan node dikunjungi: ").append(order).append("\n");
    sb.append("Jumlah node dieksplorasi: ").append(explored).append("\n");
}

```



```

        if (!path.isEmpty()) {
            sb.append(" ✨ Jalur ditemukan: ").append(String.join(" -> ", path)).append("\n");
        } else {
            sb.append("Peringatan: Tidak ada jalur dari start ke goal.\n");
        }
        resultArea_2029.setText(sb.toString());
    }

    public void resetGraph_2029() {
        graphPanel_2029.reset_2029();
        resultArea_2029.setText("");
    }

    // Inner class untuk visualisasi graph dengan background putih
    class GraphPanel_2029 extends JPanel {
        private Set<String> visited_2029;
        private List<String> path_2029;
        private String start_2029, goal_2029;

        GraphPanel_2029() {
            visited_2029 = new HashSet<>();
            path_2029 = new ArrayList<>();
        }

        void setVisitedAndPath_2029(Set<String> vis, List<String> p, String s, String g) {
            visited_2029 = new HashSet<>(vis);
            path_2029 = new ArrayList<>(p);
            start_2029 = s;
            goal_2029 = g;
            repaint();
        }

        void reset_2029() {
            visited_2029.clear();
            path_2029.clear();
            start_2029 = null;
            goal_2029 = null;
            repaint();
        }

        @Override
        protected void paintComponent(Graphics g) {
            super.paintComponent(g);
            Graphics2D g2d = (Graphics2D) g;
            g2d.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
RenderingHints.VALUE_ANTIALIAS_ON);

            // Gambar edge (jalan antar lokasi)
            g2d.setColor(Color.DARK_GRAY);

```

```

g2d.setStroke(new BasicStroke(2));
for (String u : adjList_2029.keySet()) {
    Point p1 = nodePositions_2029.get(u);
    for (String v : adjList_2029.get(u)) {
        Point p2 = nodePositions_2029.get(v);
        g2d.drawLine(p1.x, p1.y, p2.x, p2.y);
    }
}

// Gambar vertex
for (String v : vertices_2029) {
    Point pos = nodePositions_2029.get(v);
    int d = 45;
    int x = pos.x - d/2;
    int y = pos.y - d/2;

    // Warna berdasarkan status
    if (path_2029.contains(v) && !v.equals(start_2029) && !v.equals(goal_2029))
{
        g2d.setColor(new Color(0, 180, 0)); // hijau terang untuk jalur
    } else if (v.equals(start_2029)) {
        g2d.setColor(Color.RED);
    } else if (v.equals(goal_2029)) {
        g2d.setColor(Color.ORANGE);
    } else if (visited_2029.contains(v)) {
        g2d.setColor(new Color(173, 216, 230)); // biru muda untuk dikunjungi
    } else {
        g2d.setColor(new Color(255, 255, 150)); // kuning muda untuk belum
dikunjungi
    }
    g2d.fillOval(x, y, d, d);
    g2d.setColor(Color.BLACK);
    g2d.drawOval(x, y, d, d);
    g2d.setFont(new Font("SansSerif", Font.BOLD, 10));
    FontMetrics fm = g2d.getFontMetrics();
    int tw = fm.stringWidth(v);
    g2d.drawString(v, pos.x - tw/2, pos.y + 5);
}
}
}

public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> new BFSDFSMap_2511532029());
}
}

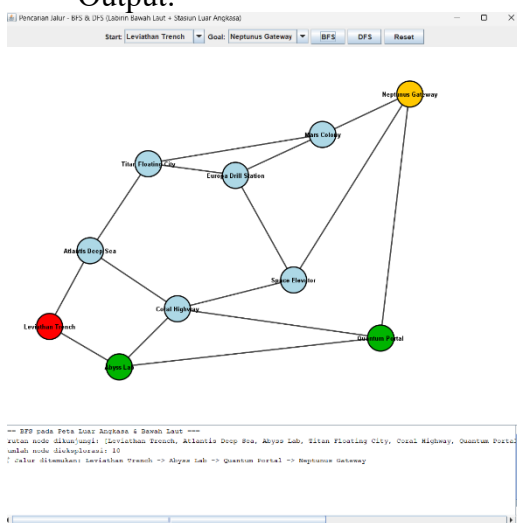
```

a) Deskripsi program BFSDFSMap_2511532029:

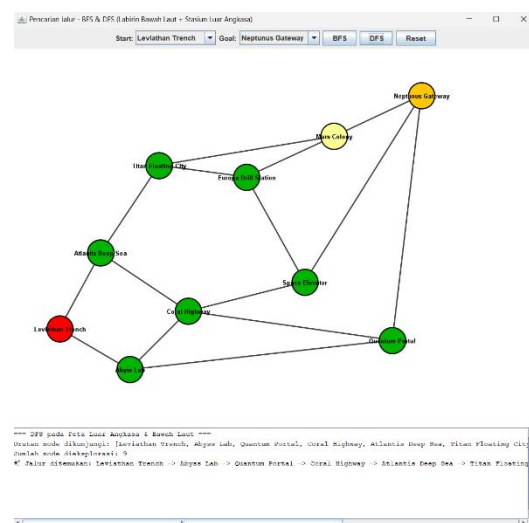
Program ini adalah aplikasi GUI lengkap yang memenuhi semua spesifikasi tugas:

- Graf : 10 vertex dengan nama-nama futuristik (Neptunus Gateway, Mars Colony, dll) yang menggabungkan tema luar angkasa dan bawah laut. Terdapat 15 edge (tak berarah) yang menghubungkan kedua dunia tersebut, misalnya "Space Elevator" menghubungkan "Neptunus Gateway" ke "Coral Highway" (transisi dari luar angkasa ke bawah laut).
- Visualisasi: Panel khusus GraphPanel2029 menggambar semua node sebagai lingkaran dengan koordinat tetap. Edge digambar sebagai garis abu-abu. Warna node berubah berdasarkan status:
 - o Merah: node start
 - o Oranye: node goal
 - o Hijau terang: node yang termasuk dalam jalur hasil (bukan start/goal)
 - o Biru muda: node yang telah dikunjungi selama penelusuran tetapi tidak termasuk jalur akhir
 - o Kuning muda: node belum dikunjungi
- Pencarian: Ketika pengguna memilih start, goal, lalu menekan tombol BFS atau DFS, program akan:
 - o Menjalankan algoritma yang sesuai (BFS menggunakan queue, DFS menggunakan stack)
 - o Mencatat urutan kunjungan (order2029) dan parent setiap node
 - o Membangun jalur dari goal ke start menggunakan parent map
 - o Menampilkan hasil di JTextArea (urutan kunjungan, jumlah node dieksplorasi, dan jalur yang ditemukan)
 - o Memperbarui visualisasi dengan memanggil graphPanel2029.setVisitedAndPath2029(visited2029, path2029, start, goal)
- Reset: Tombol reset membersihkan semua highlight dan teks hasil.

Output:



Gambar 2.7



Gambar 2.8

BAB III

KESIMPULAN

3.1 Kesimpulan

Berdasarkan praktikum minggu ke-9 tentang implementasi binary tree dan traversal graf (BFS/DFS), dapat disimpulkan bahwa struktur data pohon biner mampu merepresentasikan hubungan hierarkis secara efisien melalui simpul (node) dengan anak kiri dan kanan. Operasi-operasi dasar seperti pencarian, penghitungan jumlah simpul, serta tiga metode traversal (preorder, inorder, postorder) berhasil diimplementasikan secara rekursif dengan kode yang ringkas dan mudah dipahami. Selain itu, representasi visual pohon menggunakan indentasi memberikan gambaran struktur yang jelas. Dari sisi graf, algoritma Depth First Search (DFS) yang dijalankan secara rekursif maupun iteratif dengan stack mampu mengeksplorasi cabang sedalam mungkin sebelum backtracking, sedangkan Breadth First Search (BFS) dengan queue menjelajahi simpul level per level, sehingga BFS lebih cocok untuk menemukan jalur terpendek pada graf tak berbobot. Kedua algoritma ini memerlukan mekanisme penandaan simpul yang telah dikunjungi untuk menghindari pengulangan tak terbatas akibat adanya siklus, berbeda dengan pohon yang secara alami tidak memiliki siklus sehingga rekursi dapat berhenti pada daun.

Tugas besar visualisasi "Out of The Box: Labirin Bawah Laut & Stasiun Luar Angkasa" berhasil menggabungkan konsep BFS dan DFS dengan antarmuka grafis Java Swing, menampilkan graf berjumlah 10 simpul dan 15 sisi bertema unik. Pengguna dapat memilih simpul awal dan tujuan, lalu menjalankan algoritma untuk melihat secara visual simpul yang dikunjungi (warna biru muda) serta jalur hasil (warna hijau terang), sementara area teks mencatat urutan eksplorasi dan jalur yang ditemukan. Implementasi ini membuktikan bahwa algoritma pencarian pada graf dapat diintegrasikan ke dalam aplikasi interaktif dengan manajemen state yang baik, memberikan pengalaman belajar yang lebih intuitif dibandingkan eksekusi command line. Secara keseluruhan, praktikum pekan 9 memberikan pemahaman mendasar tentang dua struktur data non-linear yang penting, serta keterampilan untuk mengimplementasikannya dalam bentuk program console maupun GUI interaktif.

3.2 Saran

1. Pembagian materi pra-praktikum melalui iLearn tetap perlu dilanjutkan agar mahasiswa memiliki fondasi konseptual yang matang.

DAFTAR PUSTAKA

- [1] Oracle. "Trail: Creating a GUI With Swing". The Java™ Tutorials. 2024. Available:
<https://docs.oracle.com/javase/tutorial/uiswing/>
- [2] Oracle. "Lesson: Generics". The Java™ Tutorials. 2024